Задание 5. JWT Access/Refresh и управление сессиями

Реализуйте в своём приложении выдачу access- и refresh-токенов, обновление пары и серверный учёт refresh-сессий. Оба токена должны быть JWT с подписью HS256.

Реализация

1. Сущность `UserSession` и enum `SessionStatus`

Создайте сущность `UserSession` для таблицы `user_sessions` со следующими полями:

Java-поле	Java-тип	Колонка	Требования
`id`	`UUID`	`id`	Первичный ключ, генерируемый UUID
`username`	`String`	`username`	Идентификатор владельца пары, не `null`
`deviceId`	`String`	`device_id`	Метка клиента, допускается `null`
`accessToken`	`String`	`access_token`	Выданный access, тип БД `TEXT`, не `null`
`refreshToken`	`String`	`refresh_token`	Выданный refresh, тип БД `TEXT`, не `null`, уникальное значение
`accessTokenExpiry`	`Instant`	`access_token_expiry`	Время истечения access, не `null`
`refreshTokenExpiry`	`Instant`	`refresh_token_expiry`	Время истечения refresh, не `null`
`status`	`SessionStatus`	`status`	Не `null`, строковое представление enum

Создайте `SessionStatus` со значениями:

- `ACTIVE` - refresh ещё не использован и не отозван.
- `USED` - refresh успешно применён для обновления.
- `REVOKED` - refresh отозван.

Одна запись соответствует одной паре токенов. После обновления старую запись необходимо сохранить, а для новой пары создать новую строку. Время истечения в записи должно соответствовать `exp` соответствующего токена с учётом секундной точности JWT.

Подготовьте таблицу и ограничения БД. Используйте схему, принятую в своём приложении, и предоставьте сервисной роли необходимые права на чтение, добавление и изменение строк.

2. Репозиторий

Создайте репозиторий для работы с выданными токенами.
Поиск, проверка статуса, изменение старой записи и сохранение новой должны выполняться внутри одной транзакции сервиса с `@Transactional`.

3. Генерация и валидация JWT

Реализуйте класс с отдельными операциями создания и проверки access- и refresh-токенов.
Задайте настройки для определения времени жизни access- и refresh-токенов.
Для каждой выдачи создавайте новые `jti`.

Проверка каждого вида должна включать подпись, срок действия, наличие обязательных claims и соответствующее значение `type`. Access-фильтр должен принимать только `access-токены`. Обработчик обновления должен принимать только `refresh`.

4. Сервис работы с парами токенов

Реализуйте сервис, который выполняет две операции.

Выдача пары:

1. Получить пользователя после успешного входа либо создания учётной записи.
2. Создать access и refresh по требованиям выше.
3. Сохранить сессию с выданными токенами, сроками, пользователем, `deviceId` и статусом `ACTIVE`.
4. Вернуть пару только после успешного сохранения.

Обновление пары:

1. Проверить refresh как JWT и убедиться, что его `type` равен `refresh`.
2. Найти соответствующую сессию.
3. Проверить существование и допустимое состояние пользователя, соответствие `sub` владельцу записи, статус `ACTIVE` и условие `refreshTokenExpiry > now`.
4. Перевести прежнюю сессию в `USED`.
5. Создать новую пару и новую сессию `ACTIVE` с тем же пользователем и `deviceId`.
6. Завершить изменения одной транзакцией и вернуть новую пару.

Если новая пара не сохранена, изменение прежней сессии должно быть отменено. При двух одновременных обращениях с одним refresh только одно может успешно выполнить обмен.

При повторном предъявлении ещё не истёкшего refresh со статусом `USED` переведите эту запись в `REVOKED` и верните отказ без новой пары. Изменение `REVOKED` должно сохраниться, несмотря на отказ в HTTP-ответе. Технические ошибки сохранения новой пары, должны приводить к откату всей операции. Для уже отозванной сессии также возвращается отказ.

Отзыв всей цепочки обновлений и немедленный отзыв access не требуются.

5. Контроллер аутентификации и регистрация

Реализуйте следующий контракт:

Endpoint	Действие	Успешный ответ
`POST /auth/login`	Проверить логин и пароль, выдать пару токенов	`200`
`POST /auth/register`	Создать учётную запись и сразу выдать пару токенов	`201`
`POST /auth/refresh`	Обменять действующий refresh на новую пару токенов	`200`

При регистрации сохраните существующие проверки данных, уникальности учётной записи и запрет на самостоятельное получение повышенной роли. Если сохранение сессии не удалось, регистрация откатывается и не возвращает успешный ответ с токенами.

Успешный ответ каждого из трёх endpoint должен содержать пару токенов:

```
{
   "accessToken": "<JWT_ACCESS_TOKEN>",
   "refreshToken": "<JWT_REFRESH_TOKEN>"
}
```

`/auth/refresh` не должен требовать access и не принимает пароль повторно. Токены возвращайте в JSON, access передавайте в Bearer-заголовке, refresh - в JSON-теле.

Для отказов установите:

- Неверные учётные данные при входе - `401`.
- Отсутствующее или пустое обязательное поле - `400`.
- Неверный, просроченный или неподходящий по типу refresh - `401`.
- Неизвестная, использованная или отозванная refresh-сессия - `401`.
- Повторная регистрация существующей учётной записи - `409`.

Ошибочный ответ не должен содержать новую пару токенов.

Проверка результата

Обновите коллекцию запросов так, чтобы можно было воспроизвести следующий сценарий.

1. Вызвать `POST /auth/login`, получить access и refresh. В БД должна появиться первая запись со статусом `ACTIVE`.
2. С access обратиться к защищённому endpoint своего приложения и убедиться, что доступ есть.

3. Вызвать `POST /auth/refresh` с текущим refresh и получить новую пару. Проверить, что строки токенов отличаются от предыдущих. В БД первая запись должна иметь `USED`, новая - `ACTIVE`.
4. Повторно отправить старый refresh. Ожидается `401`, новая пара не выдаётся.
5. Проверить БД: первая запись имеет `REVOKED`, вторая - `ACTIVE`. Дополнительная запись после отказа не появилась.